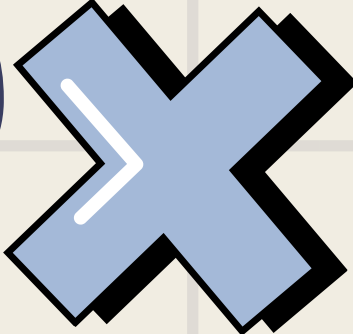


Under the guidance of
Prof. Pratap Mohanty

Machine Learning

DECISION TREES



Presented by
Lekanksha A Jain

OUTLINE

1. Intuition

Introduction to Decision Trees
Definition and Anatomy
How does a decision tree work?
Example Golf Dataset

2. Mathematics

Entropy
Gini impurity
Information Gain

3. Mechanism

Recursive Partitioning
Stopping Criteria
Concept of Overfit and Underfit
Pruning – Pre and Post

4. Coding

Hyperparameters
Steps of Model Training
Implementation (iris dataset)

1. INTUITION

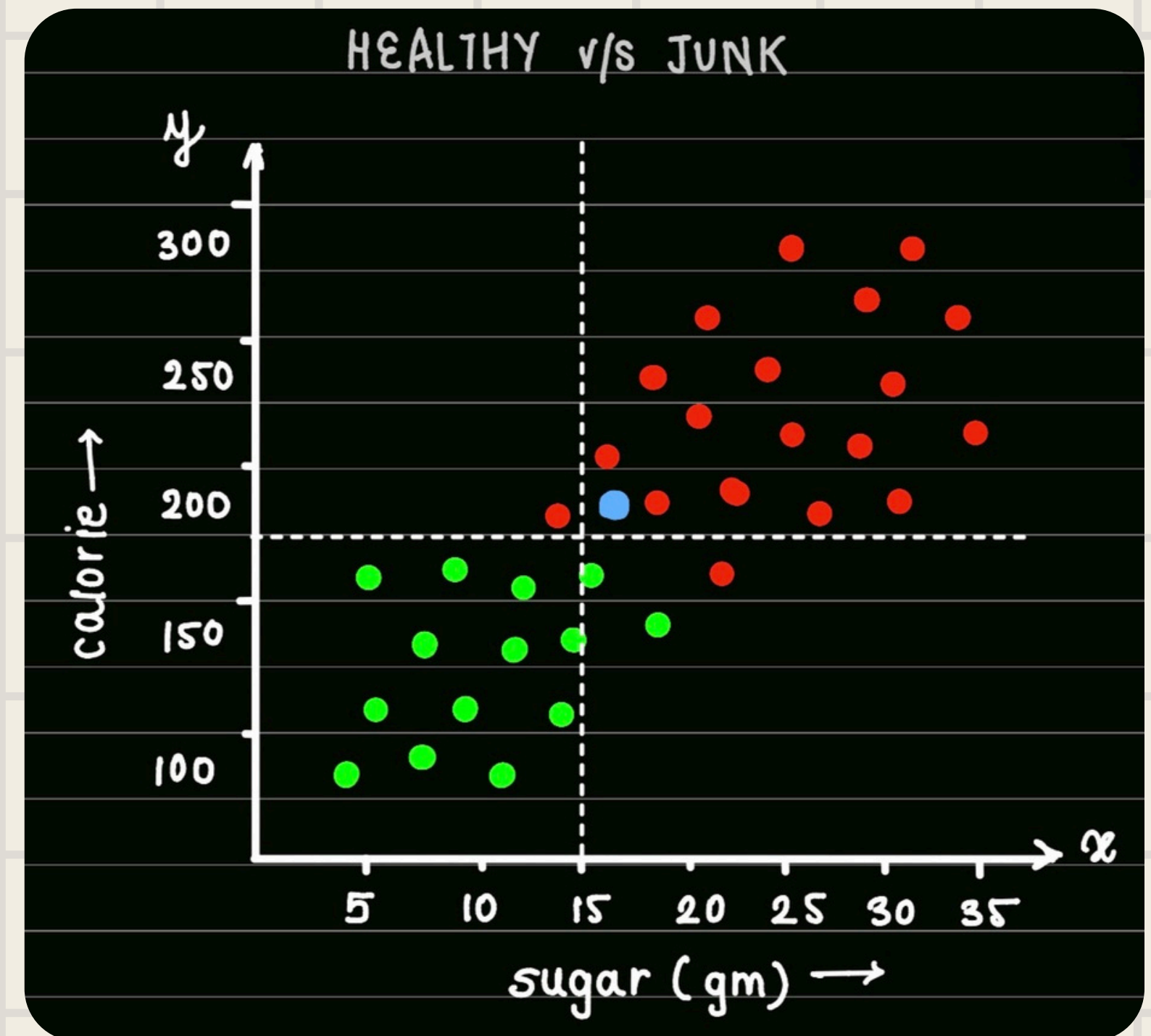
Consider the shown scatter plot where,
X axis $\rightarrow$ Sugar (g)
Y axis $\rightarrow$ Calories (kcal)

Green points:
Low Sugar + Low Calories = Healthy

Red points:
High Sugar + High Calories = Junk

Blue point signifies:
18gm of sugar and 180 calories

Can you predict whether this blue food item is Healthy or Junk?

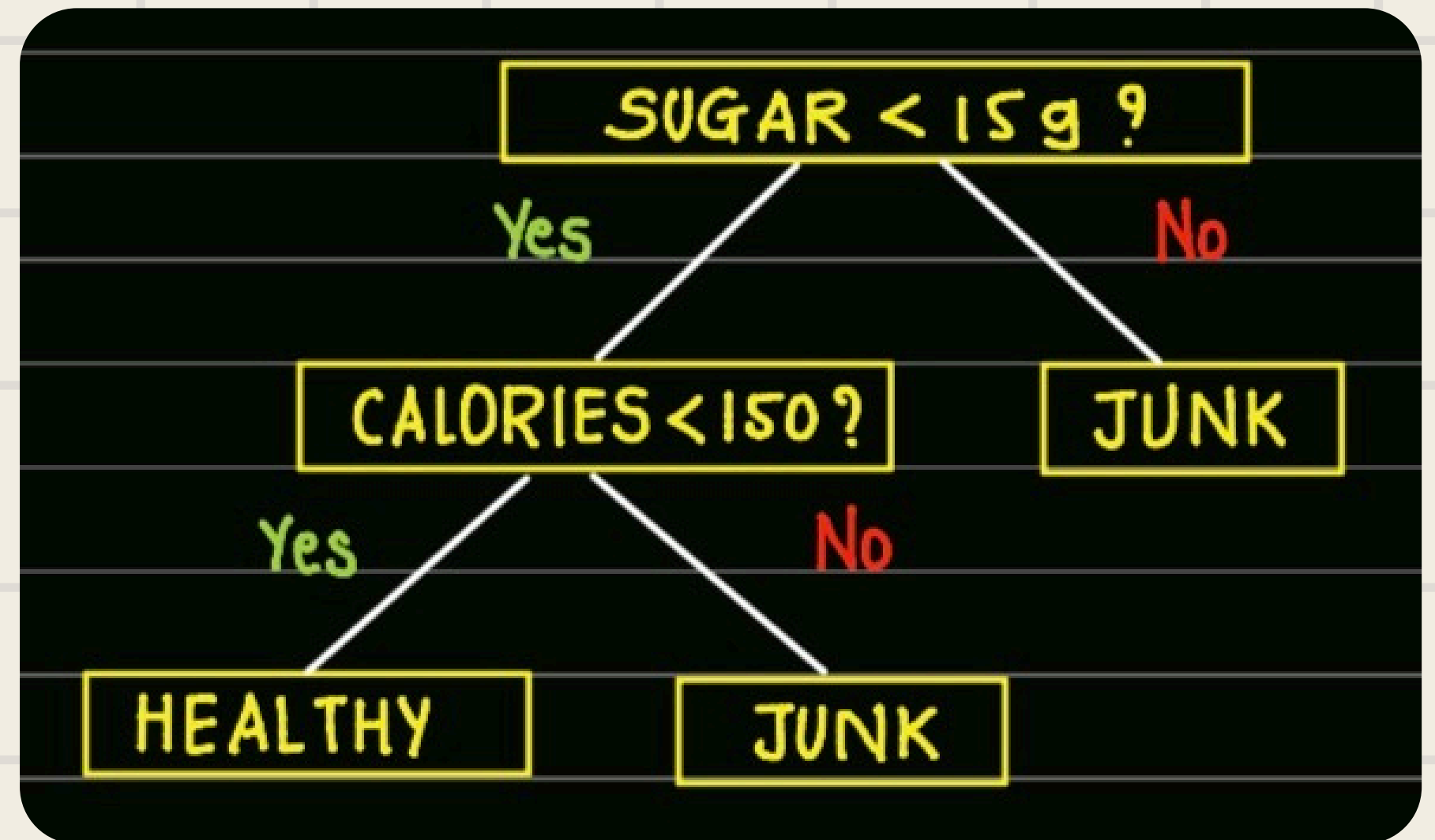


The query can be written as:

```
if sugar < 15:  
    if calories < 150:  
        print ("Healthy")  
    else:  
        print ("Junk")  
else:  
    print ("Junk")
```

It makes decisions using a sequence of if-else conditions.

Flowchart for the query:



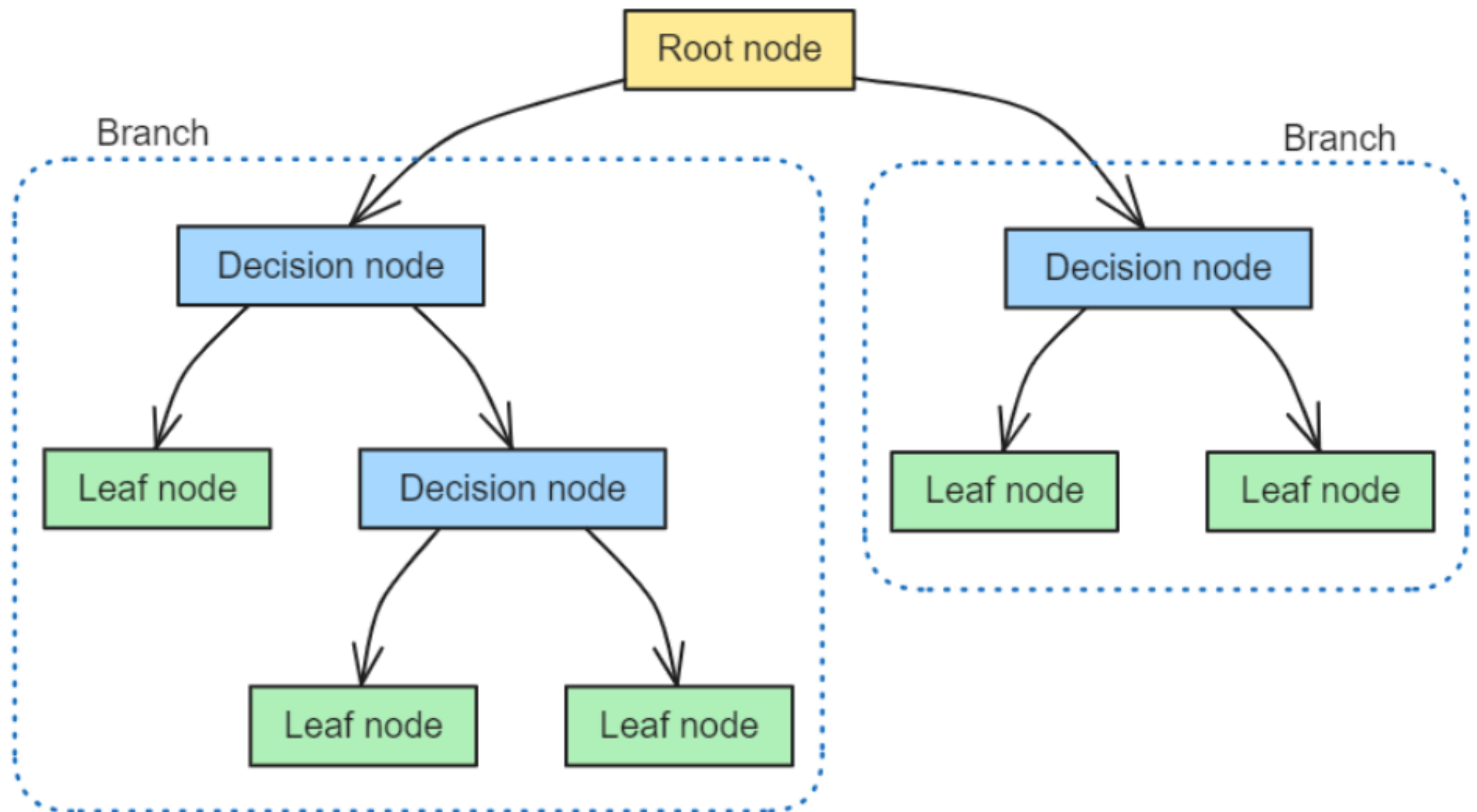
Each question splits the data into smaller groups until a final prediction is made.

Definition and anatomy

- A Decision Tree is a supervised machine learning algorithm that can be used for both:
 - (a) classification
 - (b) regression

sorts and predicts
categorical value
predicts numerical
value

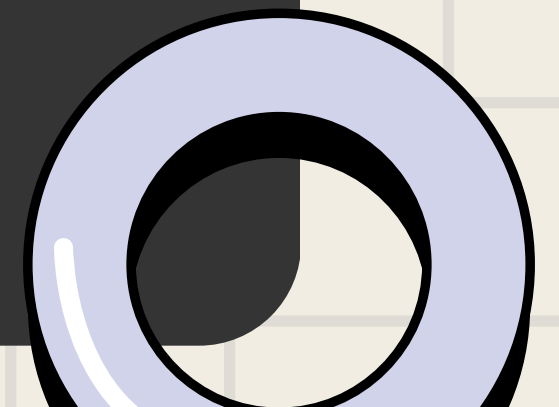
◦ SUPERVISED = model
trained using data that
already has correct answers



How does a decision tree work?

- A Decision Tree classifies data by asking a sequence of **if-else** questions.
- It begins at the **Root Node**, which contains the complete dataset.
- At each node, the algorithm selects the **best feature** to split the data (using Entropy/Gini and Information Gain).
- The process continues until a Leaf Node is reached, where the final prediction is made.
- This process is called **Recursive Partitioning**.

Let us take a look at the example!



Golf Dataset

Outlook	Temperature	Humidity	Wind	Play Golf
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Golf Dataset

Our goal is to predict whether a person will play golf (Yes/No) based on weather conditions.

Outlook	Temperature	Humidity	Wind	Play Golf
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Golf Dataset

Outlook	Temperature	Humidity	Wind	Play Golf
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Our goal is to predict whether a person will play golf (Yes/No) based on weather conditions.

Step 1: Identify Input and Output

- Input Features (Independent Variables):
 - Outlook
 - Temperature
 - Humidity
 - Wind
- Target/ Output (Dependent Variable):
 - Play Golf (Yes/No)

Outlook	Temperature	Humidity	Wind	Play Golf
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Step 2: Count the Target Values

- Yes = 9
- No = 5

Outlook	Temperature	Humidity	Wind	Play Golf
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

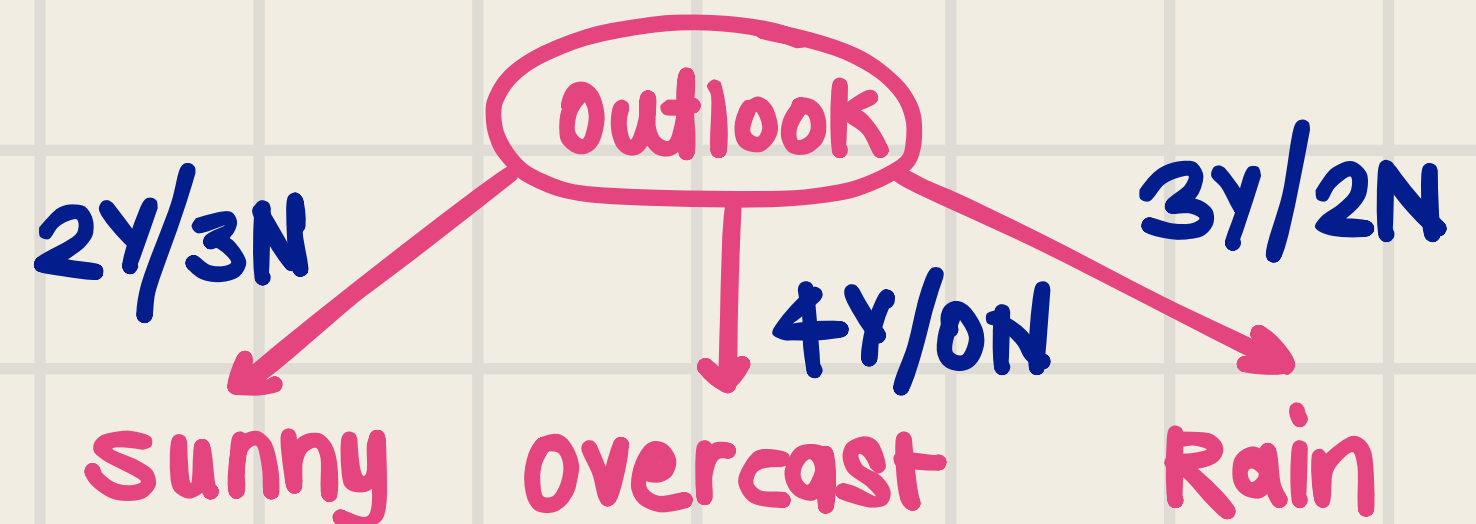
Step 2: Count the Target Values

- Yes = 9
- No = 5

Step 3: Split on a Feature

Feature chosen: Outlook.
It creates three branches:

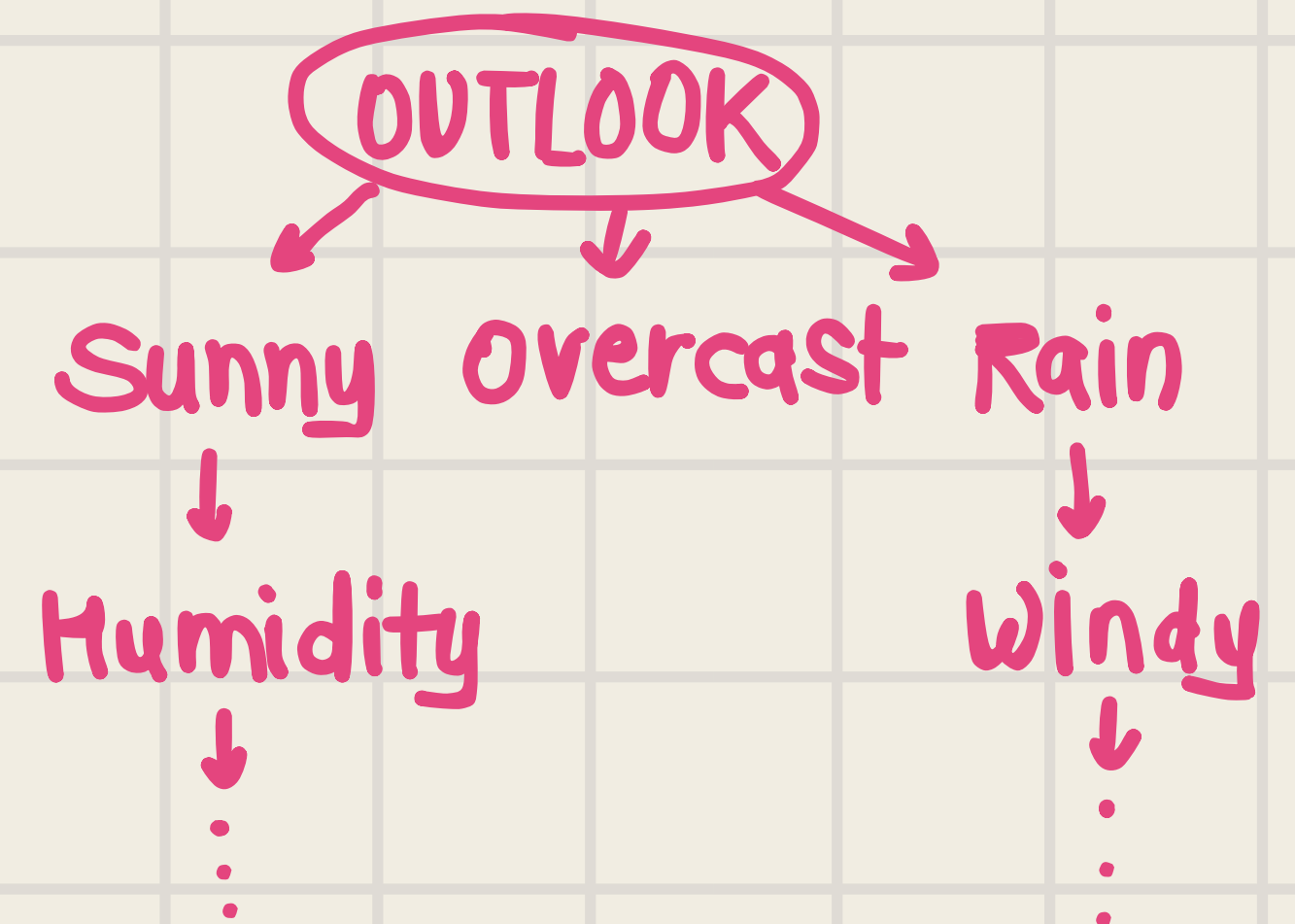
- Sunny → 2 Yes, 3 No
- Overcast → 4 Yes, 0 No
- Rain → 3 Yes, 2 No

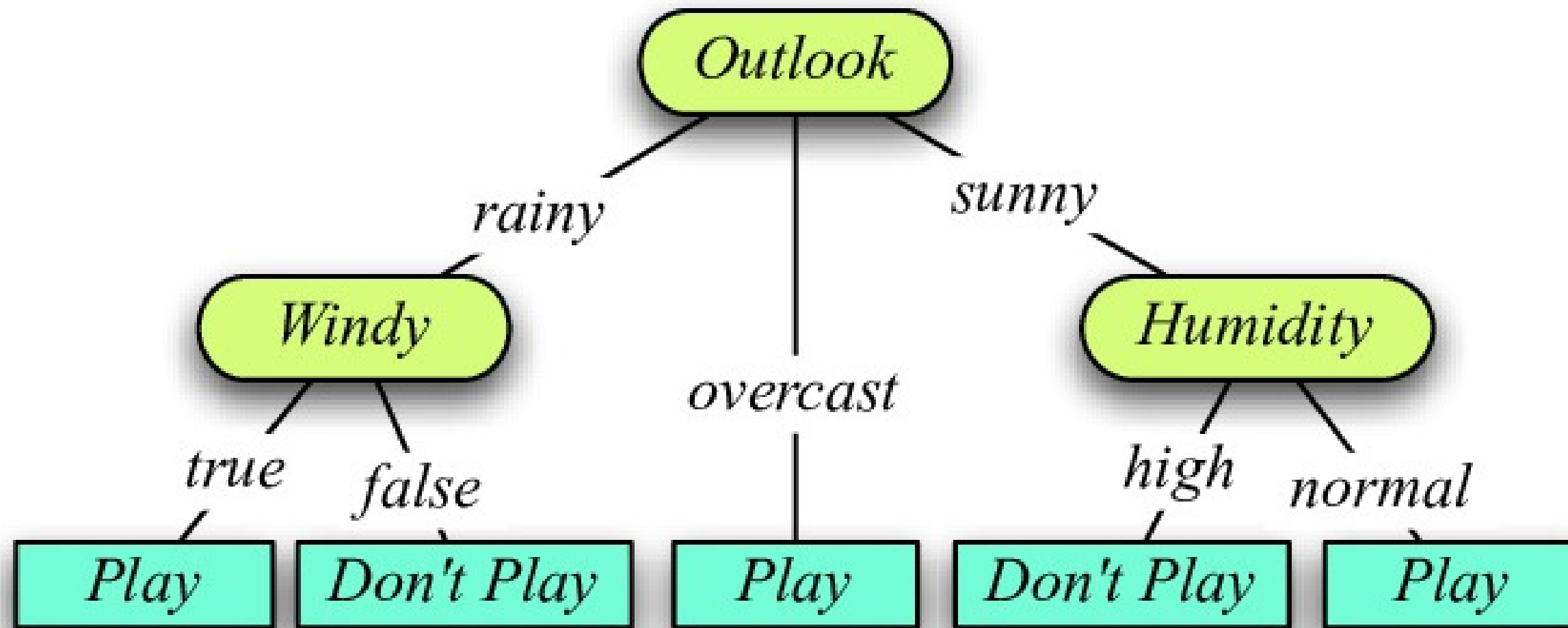


Outlook	Temperature	Humidity	Wind	Play Golf
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Step 4: Observe the Branches

- Overcast:
4 Yes, 0 No → Pure Node
- Sunny:
Mixed → Split again
- Rain:
Mixed → Split again





So the decision tree continues splitting only the impure nodes until it reaches leaf nodes.

Q. How does the computer know whether a split is pure or impure?

A. Entropy/Gini Index

Q. How does it decide which feature to split on first?

A. Information Gain

2. MATHEMATICS

01 Entropy

- Measures how mixed or uncertain a dataset is.
- Higher entropy → More impurity.
- Lower entropy → More purity.

Mathematically, FORMULA:

i. For a dataset with n classes:

where; S = dataset

$$\text{Entropy}(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

p_i = probability of class i
 $\log_2$ = logarithm to base 2

ii. For Binary Classification:

$$\text{Entropy}(S) = -p_Y \log_2 p_Y - p_N \log_2 p_N$$

p_Y = probability of YES
 p_N = probability of NO

Dataset	Entropy
3 Yes, 0 No	0 (pure node)
3 Yes, 3 No	1 (max impure)



Decision Trees try to reduce entropy after every split.

02 Gini impurity

- Measures how mixed the classes are.
- Lower Gini = Better split.
- Used by CART Decision Trees.

Mathematically, FORMULA :

i. For a dataset with n classes :

$$\text{Gini}(S) = 1 - \sum_{i=1}^n p_i^2$$

where ; S = dataset
 p_i = probability of class i

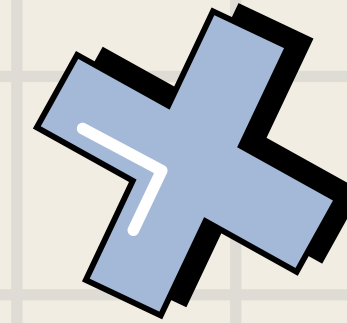
ii. For binary classification :

$$\text{Gini}(S) = 1 - (p_Y^2 + p_N^2)$$

Gini	Meaning
0	Pure
0.5	Max Impurity



03 Information Gain



- Compares all possible features.
- Chooses the feature that reduces entropy the most.
- Feature with highest Information Gain becomes the next split.

Mathematically, FORMULA:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{val}} \frac{|S_v|}{|S|} H(S_v)$$

entropy
parent node

weighted entropy
child node

; S_v = sample in
child node

S = sample in
root node

3. MECHANISM

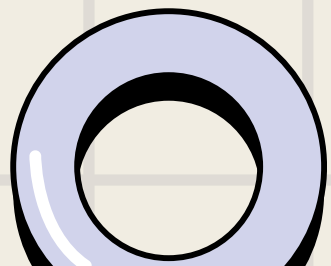
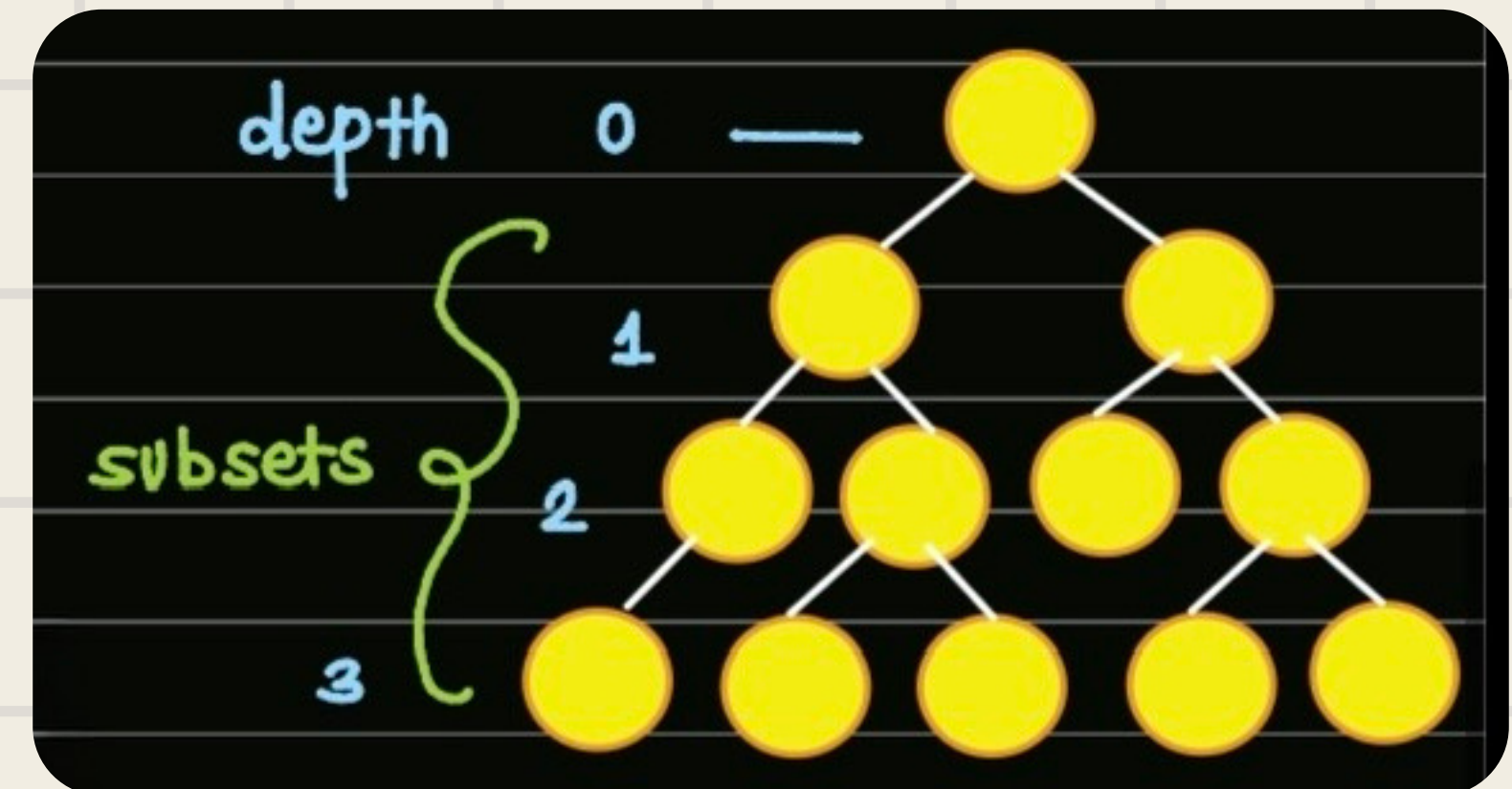


Recursive Partitioning

The process of repeatedly dividing the dataset into smaller and smaller subsets based on the best feature until a stopping condition is met.

→ STEPS:

- i. Start with complete dataset.
- ii. find best feature using Entropy / Gini.
- iii. split data into subsets.
- iv. repeat same process for each subset.
- v. continue until stopping criteria is reached.



Stopping Criteria

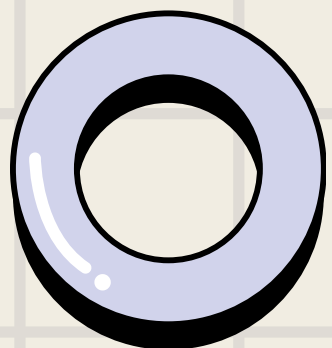
a decision tree stops growing when one of the following conditions is met:

Pure node: all samples belong to same class (no further split is needed)



Maximum depth reached: the predefined maximum depth (max-depth) is reached

Minimum samples to split: number of samples is too small to create another split (min-samples_split = 10), suppose 8 samples then can't split.



No Information Gain: further splitting does not improve the model, I.G=0

Overfit

model that learns training data too well including noise, so it performs poorly on new data.

- very complex tree
- high training accuracy
- low testing accuracy

eg. A teacher predicts students result based on marks, attendance, Study hours, assignments, project, library visits, etc.

Underfit

model that is too simple and fails to capture important patterns in the data.

- very simple tree
- low training accuracy
- low testing accuracy

eg. A teacher predicts students result only based on attendance and ignores other features like marks, assignments, etc.

Pruning

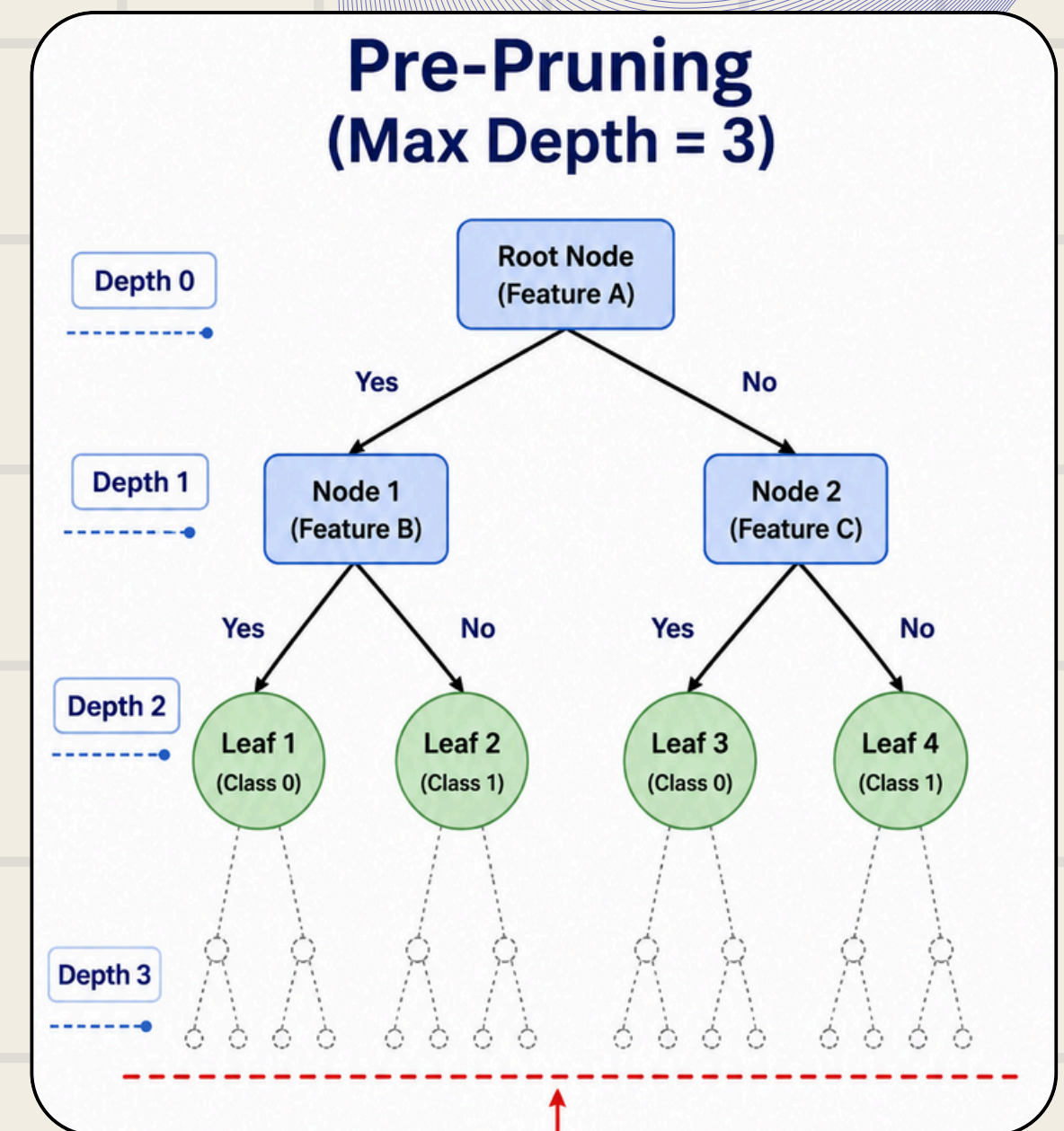
Pruning reduces the size of a decision tree by removing unnecessary branches. It helps prevent overfitting and improves generalization.

Pre-pruning

Pre-pruning stops the tree from growing further if a stopping condition is met. It prevents the tree from becoming too complex.

Common Stopping Criteria:

- `max_depth` – Maximum depth of the tree.
- `min_samples_split` – Minimum samples required to split a node.
- `min_samples_leaf` – Minimum samples required in a leaf node.
- `max_leaf_nodes` – Maximum number of leaf nodes.

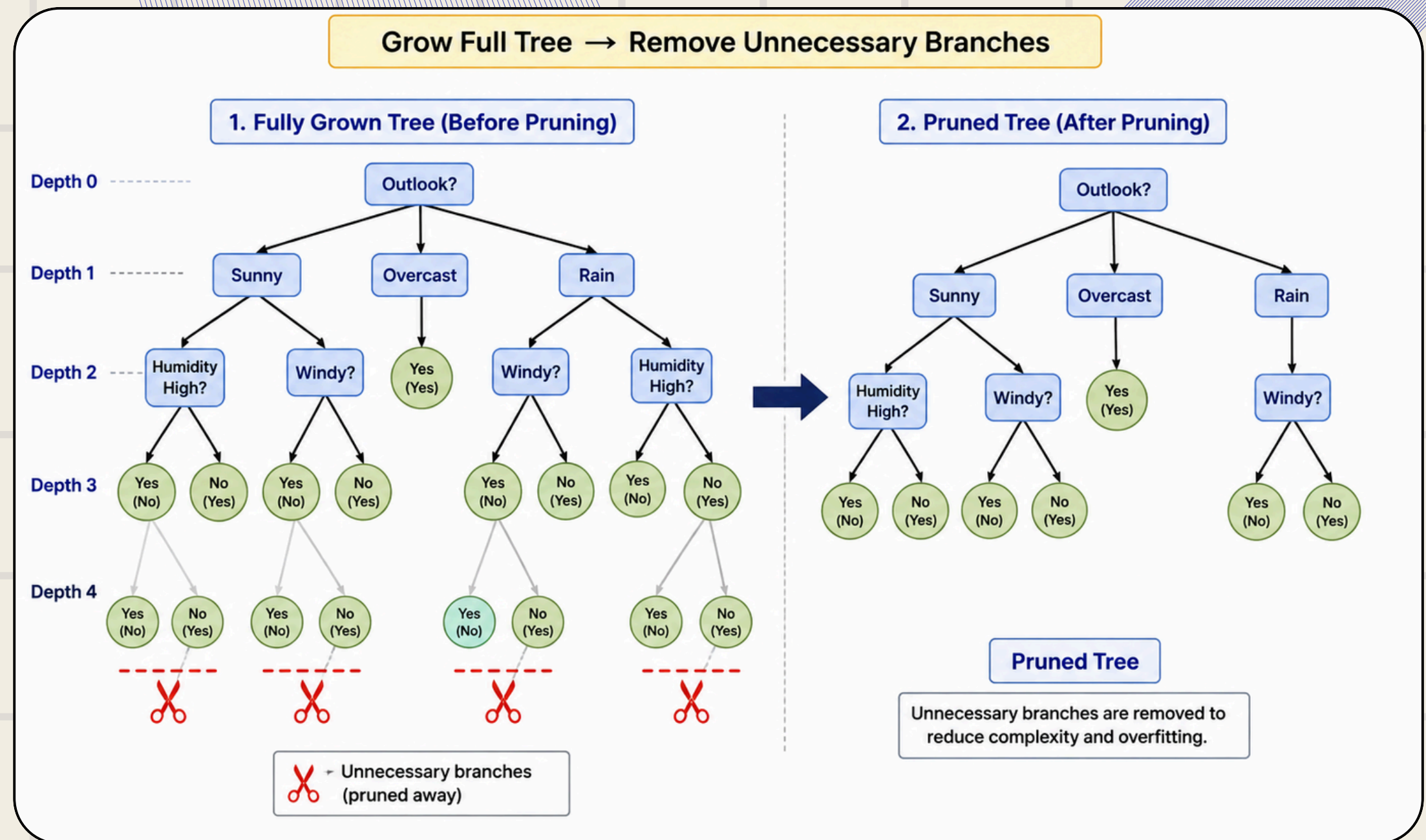


Post-pruning

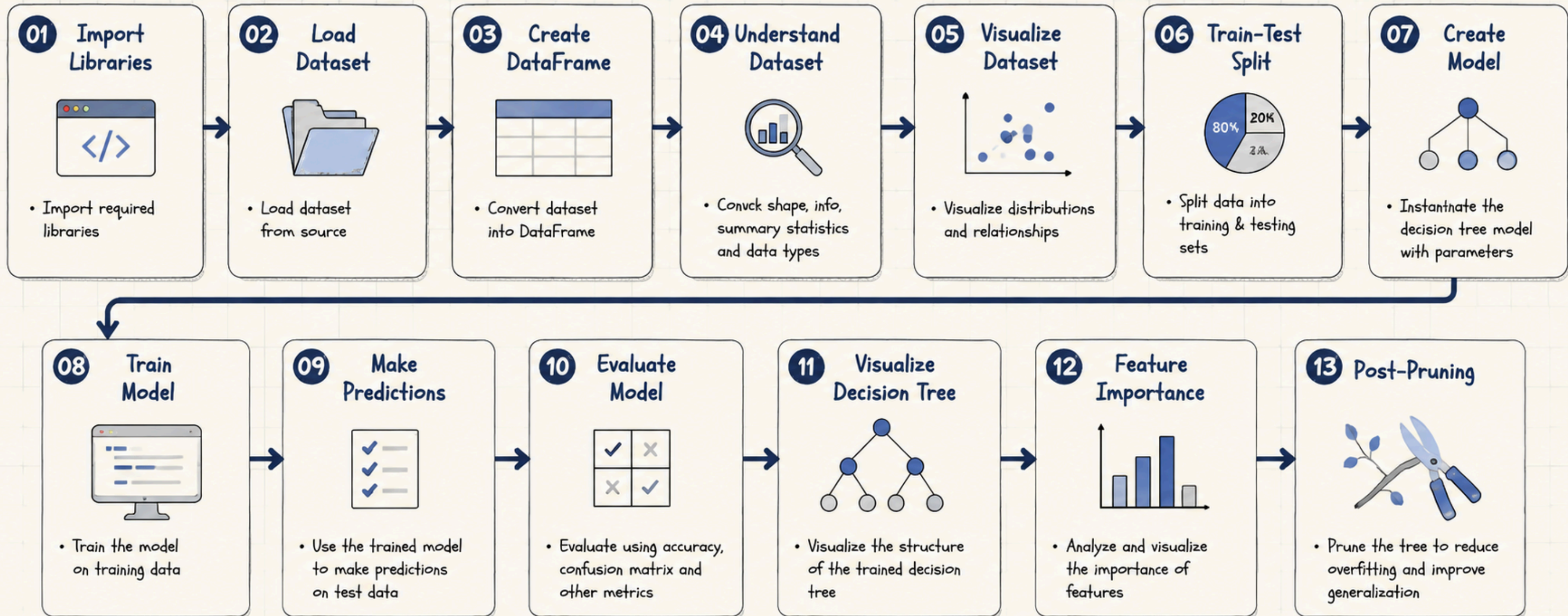
Post-pruning first builds the complete decision tree and then removes branches that do not improve prediction performance.

Parameter Used:

- `ccp_alpha` – Controls the amount of pruning.
 - Smaller value → Less pruning
 - Larger value → More pruning



4. CODING



4. CODING

01 Importing Libraries

- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

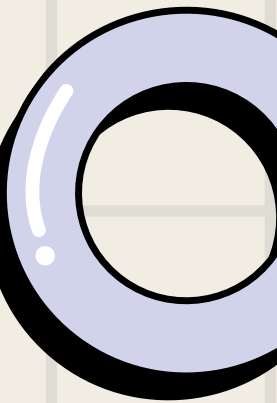
```
iris = load_iris()  
X = iris.data  
y = iris.target
```

02 Load Dataset

- $X \rightarrow$ Features/ Independent Variables/ Input
- $y \rightarrow$ Target/ Dependent Variable/ Output

03 Understand Dataset

```
df.head()  
df.shape  
df.info()  
df.describe()
```



4. CODING



04 Visualize Dataset

- Observe relationship between features
- Identify different classes visually

05 Train-test-split

- 80% Training
- 20% Testing

```
train_test_split()
```

06 Create DT Model

Use of Hyperparamters:

- criterion
- random_state
- max_depth
- ccp_alpha

```
DecisionTreeClassifier()
```

4. CODING

08 Train the model

- The model learns patterns from the training data.

```
model.fit(X_train, y_train)
```

09 Make Predictions

- Predict labels for unseen data.

```
model.predict(X_test)
```

10 Evaluate Model

- accuracy
- confusion matrix

TP	FN
FP	TN

4. CODING



11 Visualize DT

- Visualize the trained Decision Tree.
- Shows how the model makes decisions.
- Each node represents a decision (split).
- Leaf nodes represent the final predicted class.

12 Feature Importance

- Shows which features influence prediction the most.
- Higher importance = greater contribution.

13 Post Pruning

- Removes unnecessary branches.
- Reduces overfitting.
- Improves generalization on unseen data.
- Controlled using `ccp_alpha` in Scikit-learn.

THANKYOU!



Lekanksha Jain